



AGRUPAMENTO DE ESCOLAS
EDUARDO GAGEIRO

Escola Secundária de Sacavém

**CURSO PROFISSIONAL DE GESTÃO E
PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS**

PROVA DE APTIDÃO PROFISSIONAL

Relatório de Projeto

Ano Letivo 2025/2026

Relatório de Projeto

Prova de Aptidão Profissional

Mage TD



David Kouadio
2025/2026

Professoras Orientadoras: Cláudia Barata & Paula Alcobia

Índice

1 – Introdução.....	4
2 – Tecnologias Envolvidas.....	5
3 – Implementação do Projeto.....	7
3.1 – Cronograma de desenvolvimento do projeto.....	7
3.2 – Aspectos técnicos do desenvolvimento do trabalho.....	8
4 – Contexto do Projeto.....	9
4.1 – História.....	9
4.2 – Trajetória do jogo.....	10
5 – Conclusão.....	11
6 – Bibliografia/ Webgrafia.....	12
7 – Anexos.....	13

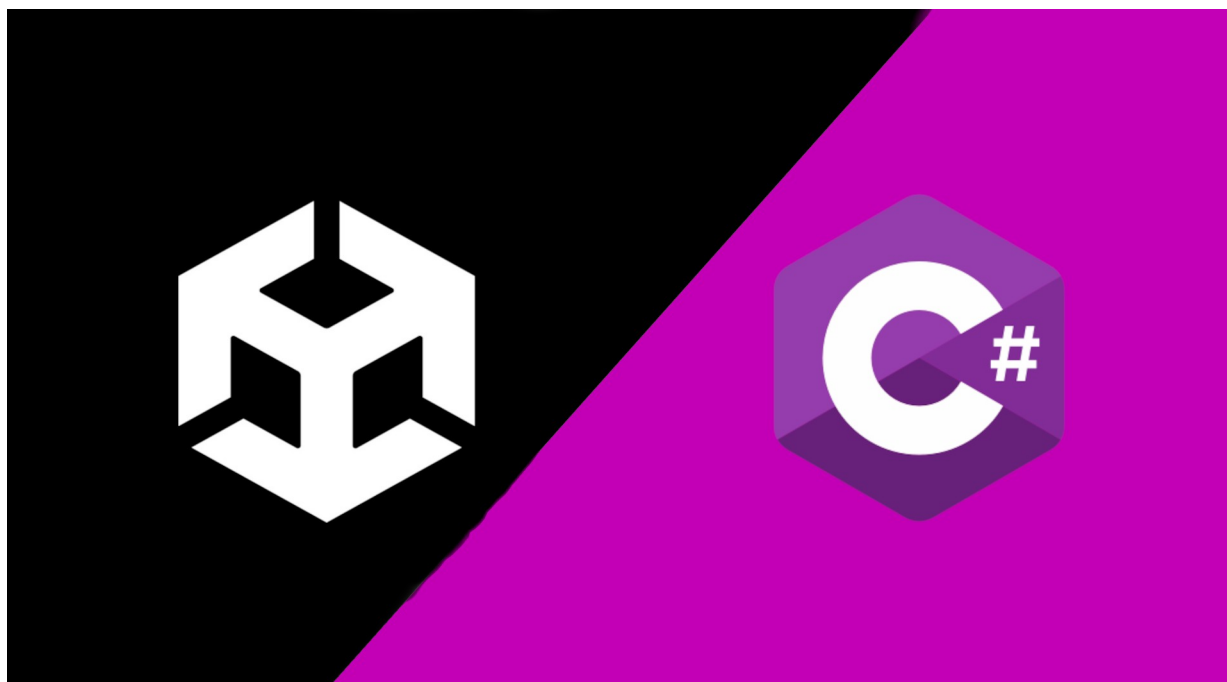
Índice de Figuras

1: Unity-C#.....	4
2 - Mapa do Mundo.....	10
3 - Reino de Aerfran.....	10

1 – Introdução

Este relatório descreve a **Prova de Aptidão Profissional (PAP)**, realizada no âmbito do Curso Profissional de Gestão e Programação de Sistemas Informáticos. O trabalho consistiu no desenvolvimento de um jogo digital do género **First-Person Shoot (FPS)**, denominado **Mage TD**, no qual o jogador recorre a magos e feitiços para avançar o rumo da história.

O projeto teve como principal objetivo a aplicação prática dos conhecimentos adquiridos ao longo do curso, especialmente nas áreas de programação e lógica computacional. Para a sua concretização, foi utilizado o motor de jogos **Unity** e a linguagem de programação **C#**. O desenvolvimento do trabalho envolveu diversas fases, incluindo a conceção da ideia, planeamento, implementação e testes. O relatório encontra-se organizado em capítulos que apresentam o enquadramento do projeto, as tecnologias utilizadas, a descrição da implementação, os aspetos técnicos relevantes e, por fim, a conclusão.



2 – Tecnologias Envolvidas

No desenvolvimento do projeto **Mage TD** foram utilizadas diversas tecnologias e ferramentas, fundamentais para a concretização do jogo:

- **Unity:** Motor de jogos utilizado para o desenvolvimento do projeto. Permitiu a criação de cenários, gestão de objetos, implementação de mecânicas de jogo e exportação para a plataforma Web.
- **C#:** Linguagem de programação principal do projeto, utilizada para implementar a lógica do jogo, controlo de inimigos, torres, sistema de ondas, pontuação e interface do utilizador.
- **Visual Studio Code:** Ambiente de desenvolvimento integrado (IDE) usado para a escrita e organização do código-fonte.
- **Git e GitHub:** Sistema de controlo de versões utilizado para gerir o código do projeto, permitindo o acompanhamento da evolução do desenvolvimento e organização dos ficheiros.
- **Unity Play:** Plataforma utilizada para publicar e disponibilizar a versão jogável do jogo online.
- **Assets gráficos e sonoros:** Recursos visuais e sonoros utilizados para enriquecer a experiência do utilizador.

- **LibreOffice:** Para a produção do relatório.
- **Paint:** Para a produção de imagens para o jogo.
- **Blender:** Para produção dos modelos 3d.
- **Youtube:** Para pesquisa.

3 – Implementação do Projeto

3.1 – Cronograma de desenvolvimento do projeto

O desenvolvimento do projeto decorreu ao longo de várias etapas:

1. Planeamento e conceção da ideia

Definição do tipo de jogo, mecânicas principais, estilo visual e objetivos do projeto.

2. Criação do projeto no Unity

Configuração inicial do ambiente de desenvolvimento e organização da estrutura do projeto.

3. Implementação das mecânicas principais

Desenvolvimento do sistema de inimigos, magos, ataques, pontos de vida e magia do jogo juntamente de todo o ambiente que o jogo ocorrerá.

4. Desenvolvimento da interface gráfica

Criação de menus, HUD, indicadores de vida e progresso do jogo.

5. Testes, correção de erros e feedback

Testes funcionais para garantir a estabilidade do jogo, correção de falhas detetadas e apreciação de críticas para o projeto.

6. Publicação da versão beta

Exportação do jogo para a plataforma Web e publicação no Unity Play.

3.2 – Aspetos técnicos do desenvolvimento do trabalho

Do ponto de vista técnico, o projeto foi desenvolvido de forma modular, recorrendo a scripts em C# para controlar os diferentes componentes do jogo.

- **Código-fonte:** Implementação de scripts responsáveis pelo movimento e ataques do jogador, inimigos, deteção de colisões e gestão objetos.
- **Lógica do jogo:** Utilização de scripts para controlar o movimento do player e o estado do jogo.
- **Interface do utilizador (UI):** Criação de elementos visuais para apresentar informações relevantes ao jogador, como vida, magia e estado da partida.
- **Layouts e cenas:** Organização das fases do jogo de forma clara, permitindo uma navegação simples entre menus e o nível principal da história.

4 – Contexto do Projeto

4.1 – História

Num mundo regido pela magia, existe um reino muito poderoso chamado Aerfran. Aerfran é um reino muito próspero e comandado por um bom rei, mas nem sempre foi assim.

Antigamente, Aerfran era um lugar caótico devido às constantes invasões de inimigos e criaturas mágicas. Para acabar com aquela situação miserável, um lendário mago daquelas terras conjurou uma muralha, cercando assim o reino. Nessa muralha, havia 4 torres para vigiar os arredores do reino: uma ao norte, sul, oeste e leste. Nas torres sul, oeste e leste ficavam os guardas e vigias, enquanto na torre do norte ficava o lendário mago, que sozinho aniquilava hordas inteiras de inimigos.

O rei da época, vendo do que um único mago era capaz, começou a incentivar o uso de magia no reino, fazendo assim com que o povo comum conseguisse ter acesso ao mínimo de magia. Claro que nem todos eram capazes de tal, mas ainda assim a integração da magia no reino foi motivo de um grande desenvolvimento.

Muito tempo se passou, e Aerfran estava cada vez mais próspero, até o falecimento do lendário mago por causas naturais. Quando essa notícia se espalhou, Aerfran, pouco tempo depois, foi severamente atacado pelo maior número de inimigos do reino e criaturas de que se tem registro. Foi aí que um jovem casal de magos, que havia recém-se mudado para Aerfran, surgiu e, com muito esforço, conseguiu impedir que o reino fosse aniquilado.

O bom rei, vendo esse esforço e sacrifício desse jovem casal, concedeu-lhes título de nobreza real. Pouco tempo se passou, e esse casal teve 4 filhos. Todos seguiram o caminho da magia, porém em áreas diferentes. Os 4 filhos se tornaram magos incomparáveis dentre todos os reinos, onde cada um deles se tornou responsável por uma torre na muralha de Aerfran.

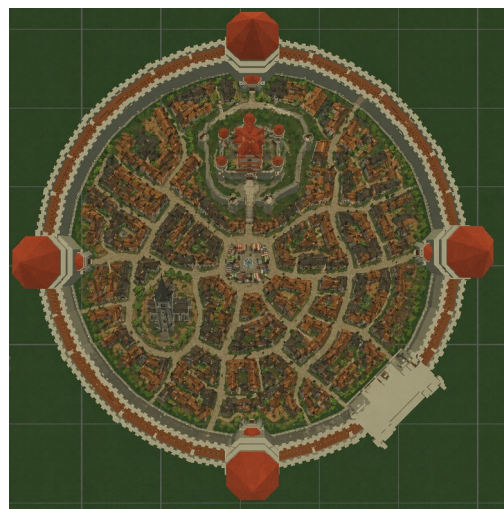
Após muitos anos de paz e tranquilidade, misteriosamente o irmão da torre oeste desapareceu juntamente com um artefato de cada mago das outras 3 torres e um artefato do lendário mago, e com isso houve uma invasão de criaturas na área oeste do reino. Após os 3 irmãos conterem a invasão, o bom rei ordenou a busca dos artefatos e do 4º irmão.

4.2 – Trajetória do jogo

Aerfran(invasão) -> "Bosques Traiçoeiros" [Pukeko] -> "Reino Bárbaro" [Mega Bárbaro] -> "Pináculo Malevolente" [Boss Final(4º irmão)]



3 - Mapa do Mundo



2 - Reino de Aerfran

5 – Conclusão

A realização deste projeto permitiu consolidar e aplicar, de forma prática, os conhecimentos adquiridos ao longo do curso, contribuindo para o desenvolvimento de competências técnicas e metodológicas.

A criação do jogo ***Mage TD*** possibilitou o aprofundamento de conhecimentos nas áreas da programação, lógica computacional, utilização de motores de jogo e resolução de problemas.

Ao longo do desenvolvimento foram encontradas diversas dificuldades, particularmente na implementação de determinadas mecânicas e na identificação e correção de erros. Estas situações foram progressivamente ultrapassadas através de pesquisa, experimentação e testes.

Como perspectivas de trabalho futuro, o projeto poderá ser expandido e melhorado mediante a introdução de novos níveis, diversificação de inimigos e armas, aperfeiçoamentos visuais e otimização do desempenho, de forma a enriquecer a experiência de utilização.

6 – BIBLIOGRAFIA/ WEBGRAFIA

As informações utilizadas para a realização tanto teórica quanto prática do projeto vieram de várias fontes, tais quais:

Tutorial para os Scripts - [https://youtube.com/playlist?](https://youtube.com/playlist?list=PLtLTOKUhgzwm1rZnTeWSRAyx9tl8VbGUE&si=I-Nee2jHBGeaDXnC)

[list=PLtLTOKUhgzwm1rZnTeWSRAyx9tl8VbGUE&si=I-Nee2jHBGeaDXnC](https://youtube.com/playlist?list=PLtLTOKUhgzwm1rZnTeWSRAyx9tl8VbGUE&si=I-Nee2jHBGeaDXnC)

Tutorial de Introdução ao Unity - <https://learn.unity.com/tutorial/get-started-with-unity-in-editor-tutorial>

Tutorial de Introdução ao C# - [https://youtube.com/playlist?](https://youtube.com/playlist?list=PLZPZq0r_RZOPNy28FDBys3GVP2LialyP_&si=oAwdRBJU7YApгуzux)

[list=PLZPZq0r_RZOPNy28FDBys3GVP2LialyP_&si=oAwdRBJU7YApгуzux](https://youtube.com/playlist?list=PLZPZq0r_RZOPNy28FDBys3GVP2LialyP_&si=oAwdRBJU7YApгуzux)

Tutorial de Introdução ao GitHub - [https://youtube.com/playlist?](https://youtube.com/playlist?list=PLODwgo8koY25mirac_ZJilnFxgf1GF9UJ&si=ZFkp-zVkr9bk6epQ)

[list=PLODwgo8koY25mirac_ZJilnFxgf1GF9UJ&si=ZFkp-zVkr9bk6epQ](https://youtube.com/playlist?list=PLODwgo8koY25mirac_ZJilnFxgf1GF9UJ&si=ZFkp-zVkr9bk6epQ)

Meu perfil no GitHub - <https://github.com/David-Kouadio>

Fonte dos Modelos 3D - <https://devassets.com/>

Fonte das Modelos 3D - <https://sketchfab.com/>

Som da bola de fogo - <https://pixabay.com/sound-effects/film-special-effects-fireball-whoosh-2-179126/>

Fonte de Modelos 3D e Partículas - <https://assetstore.unity.com/>

Fonte do overlay de dano - <https://opengameart.org/content/blood-overlay>

Fonte de textura - <https://www.magnific.com/free-photos-vectors/grass-texture>

Fonte de textura - <https://www.poliigon.com/textures/earth/grass>

Fonte de texture - https://www.reddit.com/r/Minecraft/comments/4e1yj4/110_magma_block_texture/

7 – Anexos

Código para andar e pular:

```

1  using UnityEngine;
2
3  2 references | ☒ Unity Script (2 asset references)
4  public class PlayerMotor : MonoBehaviour
5  {
6      4 references
7      private CharacterController controller;
8      5 references
9      private Vector3 playerVelocity;
10     3 references
11     private bool isGrounded;
12     1 reference | ☒ Unity Serialized Field = 5
13     public float speed = 5f;
14     2 references | ☒ Unity Serialized Field = -9.8
15     public float gravity = -9.8f;
16     1 reference | ☒ Unity Serialized Field = 1.5
17     public float jumpHeight = 3f;
18
19     // Start is called once before the first execution of Update after the MonoBehaviour is created
20     0 references | ☒ Unity Message
21     void Start()
22     {
23         controller = GetComponent<CharacterController>();
24     }
25
26     // Update is called once per frame
27     0 references | ☒ Unity Message
28     void Update()
29     {
30         isGrounded = controller.isGrounded;
31     }
32
33     //recebe a entrada de dados do InputManager.cs e aplica no character controller.
34     1 reference
35     public void ProcessMove(Vector2 input)
36     {
37         Vector3 moveDirection = Vector3.zero;
38         moveDirection.x = input.x;
39         moveDirection.z = input.y;
40         controller.Move(transform.TransformDirection(moveDirection) * speed * Time.deltaTime);
41         playerVelocity.y += gravity * Time.deltaTime;
42         if(isGrounded && playerVelocity.y < 0)
43             playerVelocity.y = -2f;
44         controller.Move(playerVelocity * Time.deltaTime);
45     }
46
47     1 reference
48     public void Jump()
49     {
50         if (isGrounded)
51         {
52             playerVelocity.y = Mathf.Sqrt(jumpHeight * -3.0f * gravity);
53         }
54     }
55 }

```

Código para controlar a ecrã:

```

1  using UnityEngine;
2
3  3 references | Unity Script (2 asset references)
4  public class PlayerLook : MonoBehaviour
5  {
6      0 references | Unity Message
7      void Start()
8      {
9          //bloquear o cursor do mouse
10         Cursor.lockState = CursorLockMode.Locked;
11     }
12
13     2 references | Unity Serialized Field = (Camera)
14     public Camera cam;
15     4 references
16     private float xRotation = 0f;
17
18     1 reference | Unity Serialized Field = 30
19     public float xSensitivity = 30f;
20     1 reference | Unity Serialized Field = 30
21     public float ySensitivity = 30f;
22
23     1 reference
24     public void ProcessLook(Vector2 input)
25     {
26         float mouseX = input.x;
27         float mouseY = input.y;
28
29         //calcular a rotação da camera olhando para cima e para baixo
30         xRotation -= (mouseY * Time.deltaTime) * ySensitivity;
31         xRotation = Mathf.Clamp(xRotation, -80f, 80f);
32
33         //aplicar para o transform da camera
34         cam.transform.localRotation = Quaternion.Euler(xRotation, 0,0);
35         //rodar a visao para esquerda e direita
36         transform.Rotate(Vector3.up * (mouseX * Time.deltaTime) * xSensitivity);
37     }
38 }

```

Código para controlar a arma e munição:

```

1  using System;
2  using System.Collections;
3  using JetBrains.Annotations;
4  using UnityEngine;
5  using UnityEngine.InputSystem;
6  using TMPPro;
7
8  7 references | ☑ Unity Script (1 asset reference)
9  public class Weapon : MonoBehaviour
10 {
11     // Portar arma
12     3 references | ☑ Unity Serialized Field = false
13     public bool isActiveWeapon;
14
15     1 reference | ☑ Unity Serialized Field = null
16     public Camera playerCamera;
17
18     //Atirar
19     5 references | 5 references | ☑ Unity Serialized Field = false | ☑ Unity Serialized Field = false
20     public bool isShooting, readyToShoot;
21     3 references
22     bool allowReset = true;
23     2 references | ☑ Unity Serialized Field = 0.2
24     public float shootingDelay = 2f;
25
26     // Rajada
27     4 references | ☑ Unity Serialized Field = 1
28     public int bulletsPerBurst = 3;
29     4 references | ☑ Unity Serialized Field = 1
30     public int burstBulletsLeft;
31
32     // Dispersão
33     4 references | ☑ Unity Serialized Field = 0
34     public float spreadIntensity;
35
36     // Bala
37     7 references
38     private PlayerInput inputActions;
39     1 reference | ☑ Unity Serialized Field = FireBallPrefab
40     public GameObject bulletPrefab;
41     2 references | ☑ Unity Serialized Field = Bullet Spawn
42     public Transform bulletSpawn;
43     1 reference | ☑ Unity Serialized Field = 100
44     public float bulletVelocity = 30;
45     1 reference | ☑ Unity Serialized Field = 3
46     public float bulletPrefabLifetime = 3f;
47
48     // Animação

```

```

35 | // Animação
36 | 2 references
37 | private Animator animator;
38 |
39 | // Carregamento
40 | 1 reference | ☑ Unity Serialized Field = 1.5
41 | public float reloadTime;
42 | 5 references | 9 references | ☑ Unity Serialized Field = 15 | ☑ Unity Serialized Field = 0
43 | public int magazineSize, bulletsLeft;
44 | 5 references | ☑ Unity Serialized Field = false
45 | public bool isReloading;
46 |
47 | // Pegar arma
48 | 3 references | ☑ Unity Serialized Field = {x: -0.36677, y: 0.4606, z: -0.3130901}
49 | public Vector3 spawnPosition;
50 | 3 references | ☑ Unity Serialized Field = {x: -126.3, y: 95.921, z: 4.547}
51 | public Vector3 spawnRotation;
52 |
53 | 5 references
54 | public enum ShootingMode
55 | {
56 |     1 reference
57 |     Single,
58 |     2 references
59 |     Burst,
60 |     1 reference
61 |     Auto
62 | }
63 |
64 | 4 references | ☑ Unity Serialized Field = 0
65 | public ShootingMode currentShootingMode;
66 |
67 | 0 references | ☑ Unity Message
68 | private void Awake()
69 | {
70 |     readyToShoot = true;
71 |     burstBulletsLeft = bulletsPerBurst;
72 |     inputActions = new PlayerInput();
73 |
74 |     bulletsLeft = magazineSize;
75 | }
76 |
77 | 0 references | ☑ Unity Message
78 | private void OnEnable()
79 | {
80 |     inputActions.Enable();
81 | }

```



```

70     }
71
72     0 references | ☐ Unity Message
73     private void OnDisable()
74     {
75         inputActions.Disable();
76     }
77
78     0 references | ☐ Unity Message
79     private void Update()
80     {
81         if(isActiveWeapon)
82         {
83             animator = WeaponManager.Instance.weaponSlots.GetComponent<Animator>();
84
85             if (bulletsLeft == 0 && isShooting)
86             {
87                 SoundManager.Instance.emptySoundfrieren.Play();
88             }
89
90             if (currentShootingMode == ShootingMode.Auto)
91             {
92                 // Segurar o botao esquerdo do rato
93                 isShooting = inputActions.OnFoot.Shoot.IsPressed();
94             }
95             else if (currentShootingMode == ShootingMode.Single || currentShootingMode == ShootingMode.Burst)
96             {
97                 // Clickar com o botao esquerdo uma vez
98                 isShooting = inputActions.OnFoot.Shoot.triggered;
99             }
100
101             if (inputActions.OnFoot.Reload.triggered && bulletsLeft < magazineSize && isReloading == false)
102             {
103                 // recarregar a munição manualmente
104                 Reload();
105             }
106
107             if(readyToShoot && !isShooting && !isReloading && bulletsLeft <= 0)
108             {
109                 // recarregar automaticamente
110                 Reload();
111             }
112
113             if (readyToShoot && isShooting && bulletsLeft > 0)
114             {
115                 burstBulletsLeft = bulletsPerBurst;
116                 FireWeapon();

```

```

116         FireWeapon();
117     }
118
119     if (inputActions.OnFoot.Reload.triggered && bulletsLeft < magazineSize && isReloading == false)
120     {
121         // recarregar a munição manualmente
122         Reload();
123     }
124
125     AmmoManager.Instance.ammo.SetActive(true);
126     AmmoManager.Instance.arm.SetActive(true);
127
128     if(AmmoManager.Instance.ammoDisplay != null)
129     {
130         AmmoManager.Instance.ammoDisplay.text = $"{bulletsLeft/bulletsPerBurst}/{magazineSize/bulletsPerBurst}";
131     }
132
133
134
135     }
136 }
137
138 1 reference
139 private void FireWeapon()
140 {
141     // diminuir munição
142     bulletsLeft--;
143
144     // instanciar animação
145     animator.SetTrigger("SHOOTING");
146
147     // instanciar som (field) AudioSource SoundManager.shootingSoundfrieren
148     SoundManager.Instance.shootingSoundfrieren.Play();
149
150     readyToShoot = false;
151
152     Vector3 shootingDirection = CalculateDirectionAndSpread().normalized;
153
154     // Instanciar a bala
155     GameObject bullet = Instantiate(bulletPrefab, bulletSpawn.position, Quaternion.identity );
156
157     // Apontar a bala para a direção do disparo
158     bullet.transform.forward = shootingDirection;
159
160     // Atirar a bala
161     bullet.GetComponent<Rigidbody>().AddForce(shootingDirection * bulletVelocity, ForceMode.Impulse);
162
163     // Destruir a bala

```

```

161
162 // Destruir a bala
163 StartCoroutine(DestroyBulletAfterTime(bullet,bulletPrefabLifeTime ));
164
165 // Checar se parou de atirar
166 if (allowReset)
167 {
168     Invoke("ResetShot",shootingDelay);
169     allowReset = false;
170 }
171
172 // Modo Rajada
173 if(currentShootingMode == ShootingMode.Burst && burstBulletsLeft > 1) // ja atirou uma vez por isso maior que 1
174 {
175     burstBulletsLeft--;
176     Invoke("FireWeapon", shootingDelay);
177 }
178
179
180
181 }
182
183 3 references
184 private void Reload()
185 {
186     SoundManager.Instance.reloadingsoundfrieren.Play();
187
188     isReloading = true;
189     Invoke("ReloadCompleted", reloadTime);
190 }
191
192 0 references
193 private void ReloadCompleted()
194 {
195     bulletsLeft = magazineSize;
196     isReloading = false;
197 }
198
199 0 references
200 private void ResetShot()
201 {
202     readyToShoot = true;
203     allowReset = true;
204 }
205
206 1 reference
207 public Vector3 CalculateDirectionAndSpread()
208 {

```



```
204 {
205     // Atirar do meio da tela para checar onde esta apontando
206     Ray ray = playerCamera.ViewportPointToRay(new Vector3(0.5f,0.5f,0));
207     RaycastHit hit;
208
209     Vector3 targetPoint;
210     if(Physics.Raycast(ray, out hit))
211     {
212         // atinjiu algo
213         targetPoint = hit.point;
214     }
215     else
216     {
217         // atirando para o ar
218         targetPoint = ray.GetPoint(100);
219     }
220
221     Vector3 direction = targetPoint - bulletSpawn.position;
222
223     float x = UnityEngine.Random.Range(-spreadIntensity, spreadIntensity);
224     float y = UnityEngine.Random.Range(-spreadIntensity, spreadIntensity);
225
226     // retorna a direção e dispersão do tiro
227     return direction + new Vector3 (x,y,0);
228 }
229
230 1 reference
231 private IEnumerator DestroyBulletAfterTime(GameObject bullet, float bulletPrefabLifTime)
232 {
233     yield return new WaitForSeconds(bulletPrefabLifTime);
234     Destroy(bullet);
235 }
236 }
```

```

1  using UnityEngine;
2
   0 references | ⓘ Unity Script (1 asset reference)
3  public class Player_bullet : MonoBehaviour
4  {
5
   0 references | ⓘ Unity Message
6  private void OnCollisionEnter(Collision objectWeHit)
7  {
8      if (objectWeHit.gameObject.CompareTag("Enemy"))
9      {
10         Debug.Log("hit " + objectWeHit.gameObject.name + " !");
11         CreateBulletImpactEffect(objectWeHit);
12         Destroy(gameObject);
13     }
14     if (objectWeHit.gameObject.CompareTag("Wall"))
15     {
16         Debug.Log("hit a wall!");
17         CreateBulletImpactEffect(objectWeHit);
18         Destroy(gameObject);
19     }
20     if (objectWeHit.gameObject.CompareTag("Ground"))
21     {
22         Debug.Log("hit the ground!");
23         CreateBulletImpactEffect(objectWeHit);
24         Destroy(gameObject);
25     }
26     if (objectWeHit.gameObject.CompareTag("ExplosiveTarget"))
27     {
28         Debug.Log("hit ExplosiveTarget!");
29         objectWeHit.gameObject.GetComponent<SphereTarget>().Shatter();
30     }
31 }
32
   3 references
33 void CreateBulletImpactEffect(Collision objectWeHit)
34 {
35     ContactPoint contact = objectWeHit.contacts[0];
36
37     GameObject hole = Instantiate(
38         GlobalReferences.Instance.bulletImpactBulletPrefab,
39         contact.point,
40         Quaternion.LookRotation(contact.normal)
41     );
42
43     hole.transform.SetParent(objectWeHit.gameObject.transform);
44
45 }
46

```

```
1  using UnityEngine;
2
   0 references | ⓘ Unity Script (2 asset references)
3  public class Bullet : MonoBehaviour
4  {
   0 references | ⓘ Unity Message
5      private void OnCollisionEnter(Collision collision)
6      {
7          Transform hitTransform = collision.transform;
8          if (hitTransform.CompareTag("Player"))
9          {
10             hitTransform.GetComponent<PlayerHealth>().TakeDamage(10);
11          }
12          Destroy(gameObject);
13      }
14  }
15
```

Código para controlar a interface gráfica:

```

1  using UnityEngine;
2  using TMPro;
3
4  2 references | ☑ Unity Script (2 asset references)
5  public class PlayerUI : MonoBehaviour
6  {
7      [SerializeField]
8      1 reference | ☑ Unity Serialized Field (multiple values)
9      private TextMeshProUGUI promptText;
10     // Start is called once before the first execution of Update after the MonoBehaviour is created
11
12     using UnityEngine;
13     using UnityEngine.UI;
14     using TMPro;
15
16     1 reference | ☑ Unity Script (2 asset references)
17     public class PlayerHealth : MonoBehaviour
18     {
19         8 references
20         private float health;
21         6 references
22         private float lerpTimer;
23         [Header("Health Bar")]
24         4 references | ☑ Unity Serialized Field = 100
25         public float maxHealth = 100f;
26         2 reference | ☑ Unity Serialized Field (multiple values)
27         public float chipSpeed = 2f;
28         3 references | ☑ Unity Serialized Field (multiple values)
29         public Image frontHealthBar;
30         5 references | ☑ Unity Serialized Field (multiple values)
31         public Image backHealthBar;
32         1 reference | ☑ Unity Serialized Field (multiple values)
33         public TextMeshProUGUI healthText;
34
35         [Header("Damage Overlay")]
36         14 references | ☑ Unity Serialized Field (multiple values)
37         public Image overlay;
38         1 reference | ☑ Unity Serialized Field = 2
39         public float duration;
40         1 reference | ☑ Unity Serialized Field = 1.5
41         public float fadespeed;
42         3 references
43         private float durationTimer;
44
45         0 references | ☑ Unity Message
46         void Start()
47         {
48             health = maxHealth;
49             overlay.color = new Color(overlay.color.r, overlay.color.g, overlay.color.b, 0);
50         }
51         0 references | ☑ Unity Message
52         void Update()
53         {
54             health = Mathf.Clamp(health, 0, maxHealth);
55             UpdateHealthUI();
56             if(overlay.color.a > 0)
57             {
58                 if(health < 30)
59                 {
60                     return;
61                 }
62                 durationTimer += Time.deltaTime;

```



```

37     durationTimer += Time.deltaTime;
38     if(durationTimer > duration)
39     {
40         //transparecer a imagem
41         float tempAlpha = overlay.color.a;
42         tempAlpha -= Time.deltaTime * fadespeed;
43         overlay.color = new Color(overlay.color.r, overlay.color.g, overlay.color.b, tempAlpha);
44     }
45 }
46
47

```

1 reference

```

48 public void UpdateHealthUI()
49 {
50     float fillF = frontHealthBar.fillAmount;
51     float fillB = backHealthBar.fillAmount;
52     float hFraction = health / maxHealth;
53     if(fillB > hFraction)
54     {
55         frontHealthBar.fillAmount = hFraction;
56         backHealthBar.color = Color.red;
57         lerpTimer += Time.deltaTime;
58         float percentComplete = lerpTimer / chipSpeed;
59         percentComplete = percentComplete * percentComplete;
60         backHealthBar.fillAmount = Mathf.Lerp(fillB, hFraction, percentComplete);
61     }
62     if(fillF < hFraction)
63     {
64         backHealthBar.color = Color.green;
65         backHealthBar.fillAmount = hFraction;
66         lerpTimer += Time.deltaTime;
67         float percentComplete = lerpTimer / chipSpeed;
68         percentComplete = percentComplete * percentComplete;
69         frontHealthBar.fillAmount = Mathf.Lerp(fillF, hFraction, percentComplete);
70     }
71
72     healthText.text = health + "/" + maxHealth;
73 }
74

```

1 reference

```

75 public void TakeDamage(float damage)
76 {
77     health -= damage;
78     lerpTimer = 0f;
79     durationTimer = 0;
80     overlay.color = new Color(overlay.color.r, overlay.color.g, overlay.color.b, 0.75f);
81 }
82

```

```

81 }
82
83 0 references
84 public void RestoreHealth(float healAmount)
85 {
86     health += healAmount;
87     lerpTimer = 0f;
88 }
89 }
90

```

Código para controle geral do projeto:

```

1  using UnityEngine;
2
3  1 reference | ☑ Unity Script (1 asset reference)
4  public class InteractionManager : MonoBehaviour
5  {
6      3 references
7      public static InteractionManager Instance { get; set; }
8
9      4 references | ☑ Unity Serialized Field = null
10     public Weapon hoveredWeapon = null;
11     1 reference | ☑ Unity Serialized Field = (Camera)
12     public Camera playerCam;
13     4 references
14     private PlayerInput inputActions;
15
16     0 references | ☑ Unity Message
17     private void OnEnable()
18     {
19         inputActions.Enable();
20     }
21
22     0 references | ☑ Unity Message
23     private void OnDisable()
24     {
25         inputActions.Disable();
26     }
27
28     0 references | ☑ Unity Message
29     private void Awake()
30     {
31         inputActions = new PlayerInput();
32
33         if (Instance != null && Instance != this)
34         {
35             Destroy(gameObject);
36         }
37         else
38         {
39             Instance = this;
40         }
41     }
42
43     0 references | ☑ Unity Message
44     void Update()
45     {
46         Ray ray = playerCam.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0));
47         RaycastHit hit;
48
49         if (Physics.Raycast(ray, out hit))
50         {

```

```
40     if(Physics.Raycast(ray, out hit))
41     {
42         GameObject objectHitByRaycast = hit.transform.gameObject;
43
44         if (objectHitByRaycast.GetComponent<Weapon>() && objectHitByRaycast.GetComponent<Weapon>().isActiveWeapon == false)
45         {
46
47             hoveredWeapon = objectHitByRaycast.gameObject.GetComponent<Weapon>();
48             hoveredWeapon.GetComponent<Outline>().enabled = true;
49
50             if (inputActions.OnFoot.Interact.triggered)
51             {
52                 WeaponManager.Instance.PickupWeapon(objectHitByRaycast.gameObject);
53             }
54         }
55         else
56         {
57             if (hoveredWeapon)
58             {
59                 hoveredWeapon.GetComponent<Outline>().enabled = false;
60             }
61         }
62     }
63
64 }
65
66
67
68
69 }
70
```

```
1  using TMPro;
2  using UnityEngine;
3
4  5 references | ⓘ Unity Script (1 asset reference)
5  public class AmmoManager : MonoBehaviour
6  {
7      7 references
8      public static AmmoManager Instance { get;set; }
9
10     // UI
11     2 references | ⓘ Unity Serialized Field = Ammo Text
12     public TextMeshProUGUI ammoDisplay;
13     1 reference | ⓘ Unity Serialized Field = Ammo
14     public GameObject ammo;
15     1 reference | ⓘ Unity Serialized Field = Left Arm
16     public GameObject arm;
17
18     0 references | ⓘ Unity Message
19     private void Awake()
20     {
21         if (Instance != null && Instance != this)
22         {
23             Destroy(gameObject);
24         }
25         else
26         {
27             Instance = this;
28         }
29     }
30 }
```

```

1  using UnityEngine;
2  using UnityEngine.InputSystem;
3
4  2 references | ⓘ Unity Script (2 asset references)
5  public class InputManager : MonoBehaviour
6  {
7      2 references
8      private PlayerInput playerInput;
9      7 references | ⓘ Unity Serialized Field
10     public PlayerInput.OnFootActions onFoot;
11
12     3 references
13     private PlayerMotor motor;
14     2 references
15     private PlayerLook look;
16
17     // Start is called once before the first execution of Update after the MonoBehaviour is created
18     0 references | ⓘ Unity Message
19     void Awake()
20     {
21         playerInput = new PlayerInput();
22         onFoot = playerInput.OnFoot;
23
24         motor = GetComponent<PlayerMotor>();
25         look = GetComponent<PlayerLook>();
26
27         onFoot.Jump.performed += ctx => motor.Jump();
28     }
29
30     // Update is called once per frame
31     0 references | ⓘ Unity Message
32     void FixedUpdate()
33     {
34         //Falar ao playermotor para mover usando os valores das ações de movimento.
35         motor.ProcessMove(onFoot.Movement.ReadValue<Vector2>());
36     }
37
38     0 references | ⓘ Unity Message
39     void LateUpdate()
40     {
41         look.ProcessLook(onFoot.Look.ReadValue<Vector2>());
42     }
43
44     0 references | ⓘ Unity Message
45     private void OnEnable()
46     {
47         onFoot.Enable();
48     }
49
50     0 references | ⓘ Unity Message
51     private void OnDisable()
52     {
53         onFoot.Disable();
54     }
55 }

```

```

1  using System;
2  using UnityEngine;
3
4  3 references | ☑ Unity Script (1 asset reference)
5  public class WeaponManager : MonoBehaviour
6  {
7      5 references
8      public static WeaponManager Instance { get; set; }
9
10     6 references | ☑ Unity Serialized Field = Player
11     public GameObject weaponslots;
12
13     3 references | ☑ Unity Serialized Field = null
14     public GameObject activeWeaponSlot;
15
16     0 references | ☑ Unity Serialized Field = null
17     public Weapon weapon;
18     //private int mode;
19
20     //if(inputs.Mode)
21
22     0 references | ☑ Unity Message
23     private void Awake()
24     {
25         if (Instance != null && Instance != this)
26         {
27             Destroy(gameObject);
28         }
29         else
30         {
31             Instance = this;
32         }
33     }
34
35     0 references | ☑ Unity Message
36     void Update()
37     {
38         if(weaponslots == activeWeaponSlot) weaponslots.SetActive(true);
39         else weaponslots.SetActive(false);
40     }
41
42     0 references | ☑ Unity Message
43     void Start()
44     {
45         activeWeaponSlot = weaponslots;
46     }
47
48     1 reference
49     public void PickupWeapon(GameObject pickedupWeapon)
50     {

```

```
1 reference
40 public void PickupWeapon(GameObject pickedupWeapon)
41 {
42     AddWeaponIntoActiveSlot(pickedupWeapon);
43 }
44
1 reference
45 private void AddWeaponIntoActiveSlot(GameObject pickedupWeapon)
46 {
47
48     pickedupWeapon.transform.SetParent(activeWeaponSlot.transform, false);
49
50     Weapon weapon = pickedupWeapon.GetComponent<Weapon>();
51
52
53     pickedupWeapon.transform.localPosition = new Vector3(weapon.spawnPosition.x, weapon.spawnPosition.y, weapon.spawnPosition.z);
54     pickedupWeapon.transform.localRotation = Quaternion.Euler(weapon.spawnRotation.x, weapon.spawnRotation.y, weapon.spawnRotation.z);
55     weaponslots.GetComponent<Animator>().enabled = true;
56     weapon.isActiveWeapon = true;
57
58 }
59
60 /*public void SwitchModes (int modeNumber)
61 {
62     if(activeWeaponSlot.transform.childCount > 0)
63     {
64         if(modeNumber % 2 == 0)
65         {
66             weapon.shootingDelay = 2;
67         }
68     }
69 }*/
70 }
71 }
72
```

Código para inimigos:

```

1  using UnityEngine;
2
3  3 references | ☑ Unity Script (3 asset references)
4  public class StateMachine : MonoBehaviour
5  {
6      10 references | ☑ Unity Serialized Field
7      public BaseState activeState;
8
9      1 reference
10     public void Initialise()
11     {
12         ChangeState(new PatrolState());
13     }
14
15     // Start is called once before the first execution of Update after the MonoBehaviour is created
16     0 references | ☑ Unity Message
17     void Start()
18     {
19     }
20
21     // Update is called once per frame
22     0 references | ☑ Unity Message
23     void Update()
24     {
25         if(activeState != null)
26         {
27             activeState.Perform();
28         }
29     }
30
31     5 references
32     public void ChangeState(BaseState newState)
33     {
34         //checar se activeState != null
35         if (activeState != null)
36         {
37             //fazer limpeza no activeState
38             activeState.Exit();
39         }
40         //mudar para novo estado
41         activeState = newState;
42
43         //checar novamente se não é null para evitar erros
44         if(activeState != null)
45         {
46             //implementar o novo estado
47             activeState.stateMachine = this;
48             activeState.enemy = GetComponent<Enemy>();
49             activeState.Enter();
50         }
51     }
52 }

```


Relatório da prova de Aptidão Profissional

```

1  using UnityEngine;
2  using UnityEngine.AI;
3
4  2 references | ⓘ Unity Script (3 asset references)
5  public class Enemy : MonoBehaviour
6  {
7      3 references
8      private StateMachine stateMachine;
9      2 references
10     private NavMeshAgent agent;
11     6 references
12     private GameObject player;
13     3 references
14     private Vector3 lastKnowPos;
15     7 references
16     public NavMeshAgent Agent {get => agent;}
17     3 references
18     public GameObject Player {get => player;}
19     2 references
20     public Vector3 LastKnowPos { get => lastKnowPos; set => lastKnowPos = value; }
21
22     2 references | ⓘ Unity Serialized Field (multiple values)
23     public Path path;
24     1 reference | ⓘ Unity Serialized Field = Sphere
25     public GameObject debugSphere;
26     [Header("Sight Values")]
27     3 references | ⓘ Unity Serialized Field = 10
28     public float sightDistance = 20f;
29     2 references | ⓘ Unity Serialized Field = 85
30     public float fieldOfView = 85f;
31     2 references | ⓘ Unity Serialized Field = 1.4
32     public float eyeHeight;
33     [Header("Weapon Values")]
34     1 reference | ⓘ Unity Serialized Field = Gunbarrel
35     public Transform gunBarrel;
36     [Range(0.1f,10f)]
37     1 reference | ⓘ Unity Serialized Field = 1.21
38     public float fireRate;
39     1 reference | ⓘ Unity Serialized Field = 40
40     public int bulletSpeed;
41     0 references | ⓘ Unity Serialized Field = 1
42     public int damage;
43     //apenas para debug
44     [SerializeField]
45     1 reference | ⓘ Unity Serialized Field =
46     private string currentState;
47
48     0 references | ⓘ Unity Message
49     void Start()
50     {
51         stateMachine = GetComponent<StateMachine>();
52         agent = GetComponent<NavMeshAgent>();
53         stateMachine.Initialise();
54         player = GameObject.FindGameObjectWithTag("Player");
55     }
56
57     // Update is called once per frame
58     0 references | ⓘ Unity Message
59     void Update()
60     {
61         CanSeePlayer();
62         currentState = stateMachine.activeState.ToString();

```

```

39 void Update()
40 {
41     CanSeePlayer();
42     currentState = stateMachine.activeState.ToString();
43     debugSphere.transform.position = lastKnowPos;
44 }
45 4 references
46 public bool CanSeePlayer()
47 {
48     if(player != null)
49     {
50         //o jogador está perto o suficiente para ser visto?
51         if(Vector3.Distance(transform.position, player.transform.position) < sightDistance)
52         {
53             Vector3 targetDirection = player.transform.position - transform.position - (Vector3.up * eyeHeight);
54             float angleToPlayer = Vector3.Angle(targetDirection, transform.forward);
55             if(angleToPlayer >= -fieldOfView && angleToPlayer <= fieldOfView)
56             {
57                 Ray ray = new Ray(transform.position + (Vector3.up * eyeHeight), targetDirection);
58                 RaycastHit hitInfo = new RaycastHit();
59                 if(Physics.Raycast(ray, out hitInfo, sightDistance))
60                 {
61                     if(hitInfo.transform.gameObject == player)
62                     {
63                         Debug.DrawRay(ray.origin, ray.direction * sightDistance);
64                         return true;
65                     }
66                 }
67             }
68         }
69     }
70     return false;
71 }
72 }
73 }
74

```

```

{
  0 references
  public void OnDrawGizmosSelected()
  {
    if (alwaysDrawPath)
    {
      return;
    }
    else
    {
      DrawPath();
    }
  }
}
#endif
}
```

```

1  using UnityEditor;
2  using UnityEngine;
3
4  2 references
5  public class AttackState : BaseState
6  {
7      3 references
8      private float moveTimer;
9      3 references
10     private float losePlayerTimer;
11     3 references
12     private float shotTimer;
13
14     2 references
15     public override void Enter()
16     {
17
18     }
19
20     2 references
21     public override void Exit()
22     {
23
24     }
25
26     2 references
27     public override void Perform()
28     {
29         if (enemy.CanSeePlayer()) //consegue ver o player
30         {
31             //trava o temporizador de perder o player de vista e incrementa o temporizador de mover e atirar
32             losePlayerTimer = 0;
33             moveTimer += Time.deltaTime;
34             shotTimer += Time.deltaTime;
35             enemy.transform.LookAt(enemy.Player.transform);
36             //se o temporizador de tiro > cadencia de tiro
37             if(shotTimer > enemy.fireRate)
38             {
39                 Shoot();
40             }
41             //move o inimigo para uma posição aleatoria depois de um tempo aleatorio
42             if(moveTimer > Random.Range(3, 7))
43             {
44                 enemy.Agent.SetDestination(enemy.transform.position + (Random.insideUnitSphere * 5));
45                 moveTimer = 0;
46             }
47             enemy.LastKnowPos = enemy.Player.transform.position;
48         }
49         else //perdeu o player de vista
50         {
51             losePlayerTimer += Time.deltaTime;
52             if(losePlayerTimer > Random.Range(3f, 8f))
53             {
54                 //mudar para o estado de patrulha
55                 stateMachine.ChangeState(new SearchState());
56             }
57         }
58     }
59 }

```

Relatório da prova de Aptidão Profissional

```
--
53
54 1 reference
55 private void Shoot()
56 {
57     //Armazenar uma referencia para o escopo
58     Transform gunbarrel = enemy.gunBarrel;
59
60     //instanciar uma nova bala
61     GameObject bullet = GameObject.Instantiate(Resources.Load("Bullets/Enemies/Pukeko/PukekoProjectilePrefab") as GameObject, gunbarrel.position, enemy.transform.rotation);
62
63     //calcular a direção para o player
64     Vector3 shootDirection = (enemy.Player.transform.position - gunbarrel.transform.position).normalized;
65
66     //adicionar força para o projétil + taxa de erro
67     bullet.GetComponent<Rigidbody>().linearVelocity = Quaternion.AngleAxis(Random.Range(-2f,3f),Vector3.up) * shootDirection * enemy.bulletSpeed;
68     shotTimer = 0;
69 }
70
71 // Start is called once before the first execution of Update after the MonoBehaviour is created
72 0 references
73 void Start()
74 {
75 }
76
77 // Update is called once per frame
78 0 references
79 void Update()
80 {
81 }
82 }
```

```

1  using UnityEngine;
2
3  1 reference
4  public class SearchState : BaseState
5  {
6      2 references
7      private float searchTimer;
8      3 references
9      private float moveTimer;
10     2 references
11     public override void Enter()
12     {
13         enemy.Agent.SetDestination(enemy.LastKnowPos);
14     }
15     2 references
16     public override void Perform()
17     {
18         if (enemy.CanSeePlayer())
19         {
20             stateMachine.ChangeState(new AttackState());
21         }
22
23         if (enemy.Agent.remainingDistance < enemy.Agent.stoppingDistance)
24         {
25             searchTimer += Time.deltaTime;
26             moveTimer += Time.deltaTime;
27
28             //move o inimigo para uma posição aleatoria depois de um tempo aleatorio
29             if(moveTimer > Random.Range(3, 5))
30             {
31                 enemy.Agent.SetDestination(enemy.transform.position + (Random.insideUnitSphere * 10));
32                 moveTimer = 0;
33             }
34
35             if(searchTimer > Random.Range(6f, 10f))
36             {
37                 stateMachine.ChangeState(new PatrolState());
38             }
39         }
40     }
41     2 references
42     public override void Exit()
43     {
44     }
45 }

```

```

1  using UnityEngine;
2
3  2 references
4  public class PatrolState : BaseState
5  {
6      //verificar qual ponto de caminho o inimigo está marcando
7      4 references
8      public int waypointIndex;
9      3 references
10     public float waitTimer;
11     2 references
12     public override void Enter()
13     {
14     }
15     2 references
16     public override void Perform()
17     {
18         PatrolCycle();
19         if (enemy.CanSeePlayer())
20         {
21             stateMachine.ChangeState(new AttackState());
22         }
23     }
24     2 references
25     public override void Exit()
26     {
27     }
28
29     1 reference
30     public void PatrolCycle()
31     {
32         //definir a logica da patrulha
33         if(enemy.Agent.remainingDistance < 0.2f)
34         {
35             waitTimer += Time.deltaTime;
36             if(waitTimer > 3)
37             {
38                 if(waypointIndex < enemy.path.waypoints.Count - 1)
39                 {
40                     waypointIndex++;
41                 }
42                 else
43                 {
44                     waypointIndex = 0;
45                     enemy.Agent.SetDestination(enemy.path.waypoints[waypointIndex].position);
46                     waitTimer = 0f;
47                 }
48             }
49         }
50     }
51 }

```

```

1      5 references
2      public abstract class BaseState
3      {
4          25 references
5          public Enemy enemy;
6          5 references
7          public StateMachine stateMachine;
8          4 references
9          public abstract void Enter();
10         4 references
11         public abstract void Perform();
12         4 references
13         public abstract void Exit();
14     }

```